

ASSIGNMENT-8.5

AI ASSISTANT CODING

NAME: B. Akhira Nandhini

Htno:2303A51516

Batch:22

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.
- Requirements:
 - o Username length must be between 5 and 15 characters.
 - o Must contain only alphabets and digits.
 - o Must not start with a digit.
 - o No spaces allowed.

Example Assert Test Cases:

```
assert is_valid_username("User123") == True
```

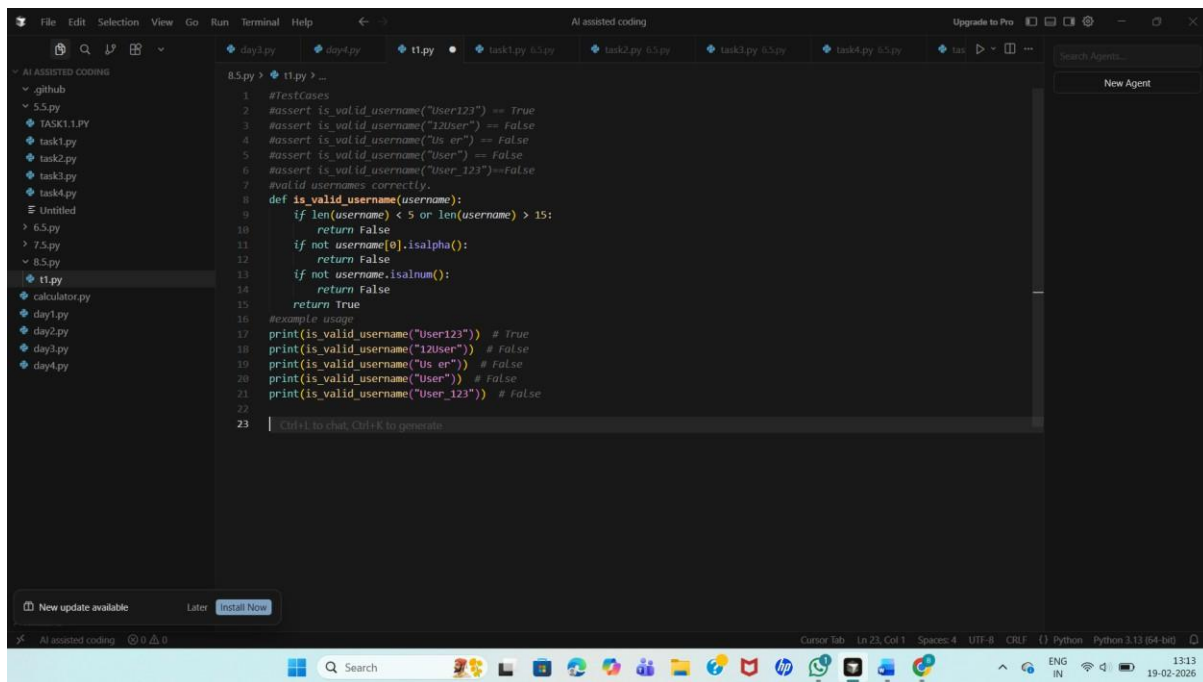
```
assert is_valid_username("12User") == False
```

```
assert is_valid_username("Us er") == False
```

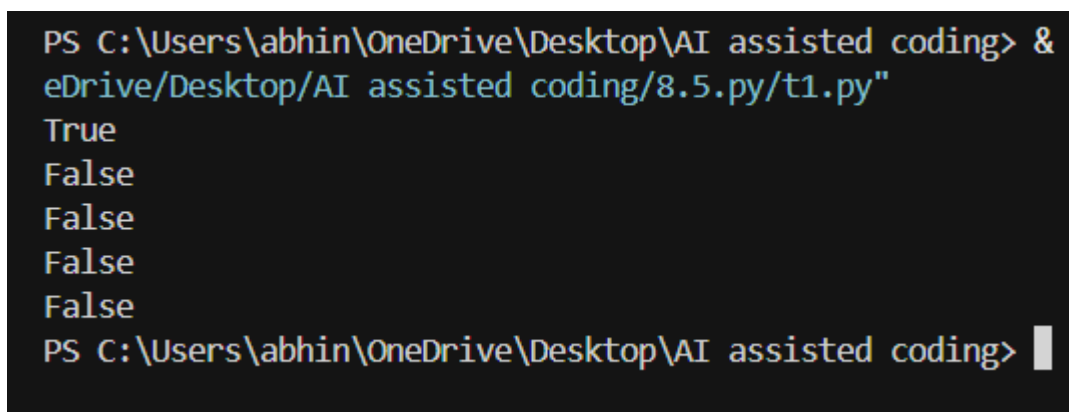
Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.

Code:



Output:



Observation: AI-generated assert test cases helped define the username validation rules before coding. By writing tests first, the function was implemented to satisfy all constraints such as length limits, allowed characters, and starting character rules. This ensured the function was reliable and handled invalid usernames correctly.

Task Description #2 (Even–Odd & Type Classification – Apply

AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

- Requirements:

- o If input is an integer, classify as "Even" or "Odd".

- o If input is 0, return "Zero".

- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

```
assert classify_value(8) == "Even"
```

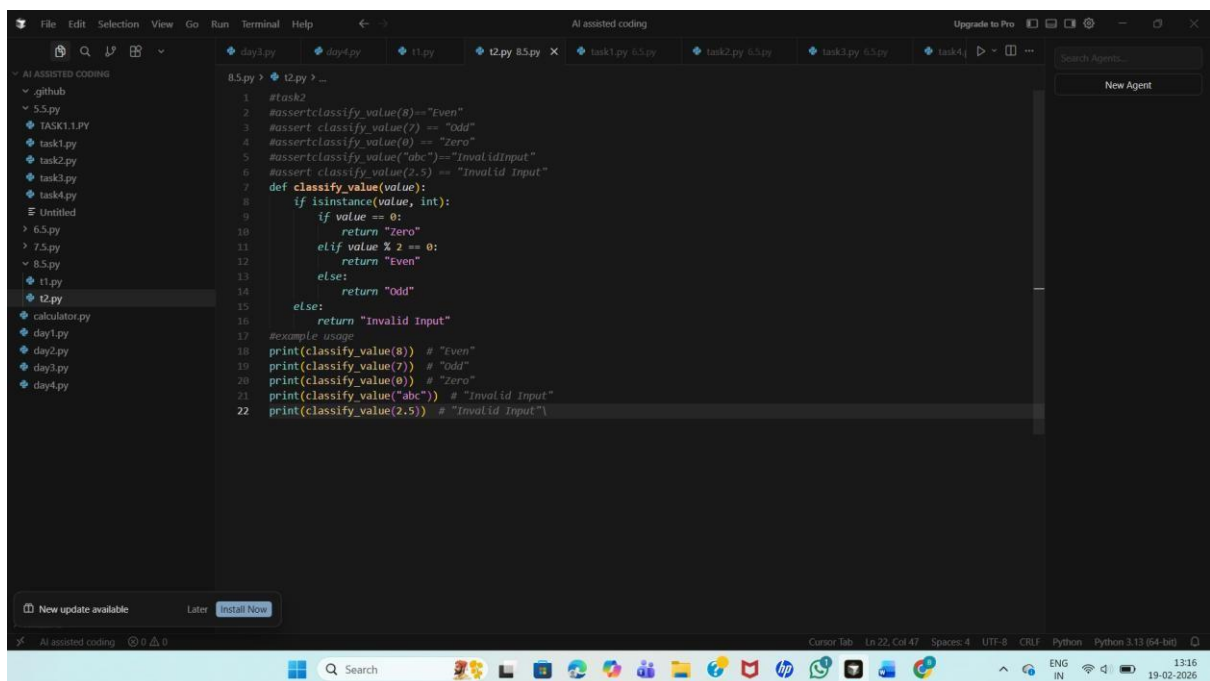
```
assert classify_value(7) == "Odd"
```

```
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- Function correctly classifying values and passing all test cases.

Code:



The screenshot shows a code editor with a dark theme. The left sidebar displays a file explorer with a tree view containing files like .github, 5.5.py, TASK1.1.PY, task1.py, task2.py, task3.py, task4.py, Untitled, 6.5.py, 7.5.py, 8.5.py, t1.py, t2.py, calculator.py, day1.py, day2.py, day3.py, and day4.py. The main editor area shows the code for t2.py, which includes a function classify_value and several test cases using assert. The code is as follows:

```
1 #task2
2 #assert classify_value(8)=="Even"
3 #assert classify_value(7) == "Odd"
4 #assert classify_value(0) == "Zero"
5 #assert classify_value("abc")=="Invalid Input"
6 #assert classify_value(2.5) == "Invalid Input"
7 def classify_value(value):
8     if isinstance(value, int):
9         if value == 0:
10             return "Zero"
11         elif value % 2 == 0:
12             return "Even"
13         else:
14             return "Odd"
15     else:
16         return "Invalid Input"
17
18 #example usage
19 print(classify_value(8)) # "Even"
20 print(classify_value(7)) # "Odd"
21 print(classify_value(0)) # "Zero"
22 print(classify_value("abc")) # "Invalid Input"
23 print(classify_value(2.5)) # "Invalid Input"
```

The bottom status bar indicates the cursor is at line 22, column 47, with 4 spaces, UTF-8 encoding, and CRLF line endings. The system tray at the bottom shows the date and time as 19-02-2026, 13:16.

Output:

```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C
eDrive/Desktop/AI assisted coding/8.5.py/t2.py"
Even
Odd
Zero
Invalid Input
Invalid Input
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> |
```

Observation: AI-assisted test cases guided the classification of different inputs such as integers, zero, and non-numeric values. The function correctly used conditional logic to identify even, odd, zero, and invalid inputs, improving robustness and error handling.

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.

- Requirements:

- o Ignore case, spaces, and punctuation.

- o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

```
assert is_palindrome("Madam") == True
```

```
assert is_palindrome("A man a plan a canal Panama") ==
```

```
True
```

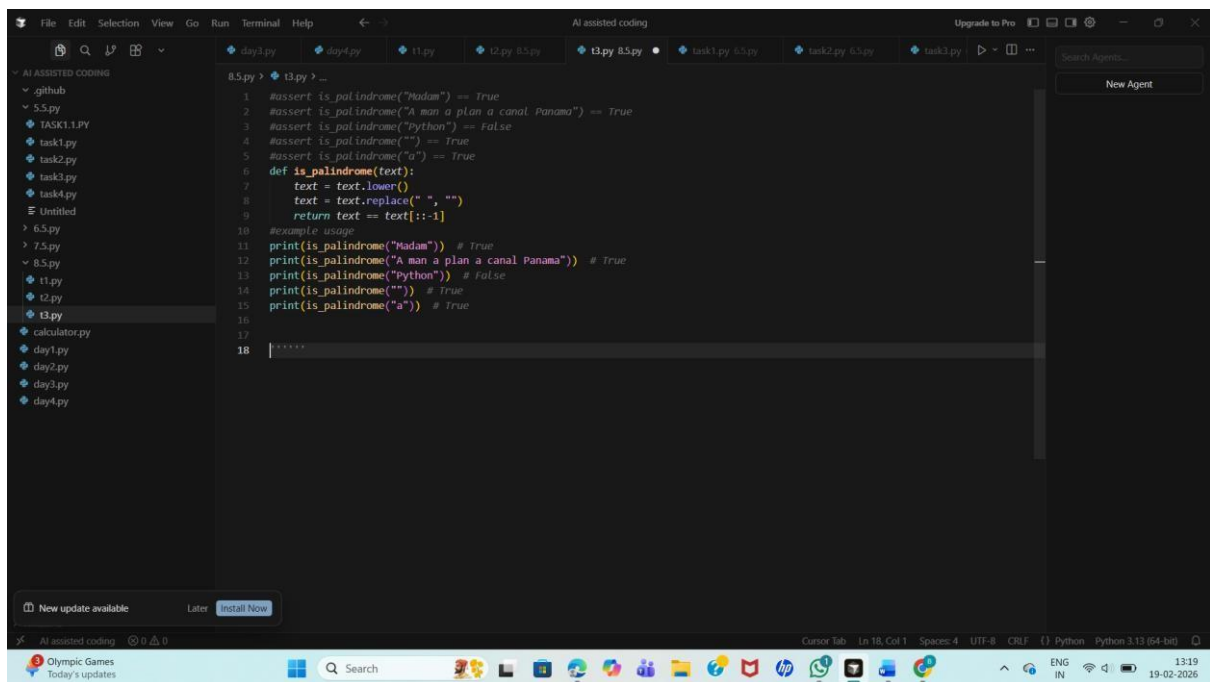
```
assert is_palindrome("Python") == False
```

Expected Output #3:

- Function correctly identifying palindromes and passing all

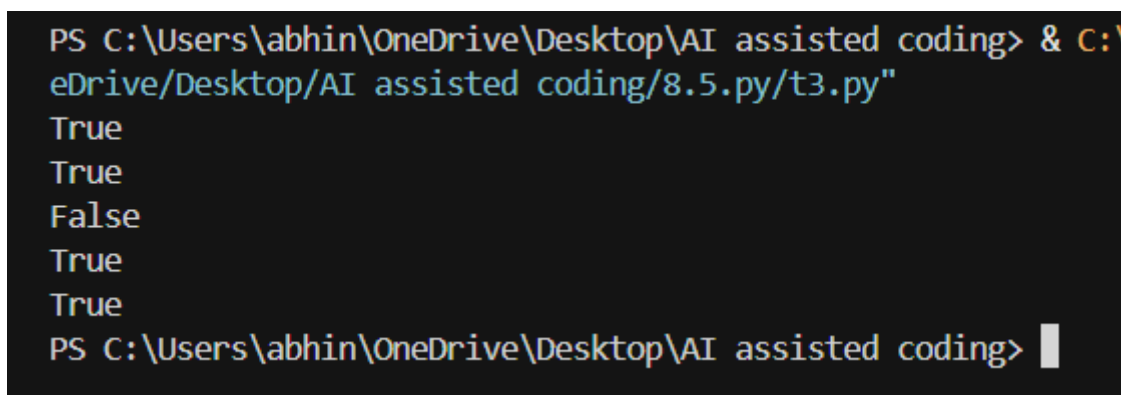
AI-generated tests.

Code:



```
1  #assert is_palindrome("Madam") == True
2  #assert is_palindrome("A man a plan a canal Panama") == True
3  #assert is_palindrome("python") == False
4  #assert is_palindrome("") == True
5  #assert is_palindrome("a") == True
6  def is_palindrome(text):
7      text = text.lower()
8      text = text.replace(" ", "")
9      return text == text[::-1]
10 #example usage
11 print(is_palindrome("Madam")) # True
12 print(is_palindrome("A man a plan a canal Panama")) # True
13 print(is_palindrome("python")) # False
14 print(is_palindrome("")) # True
15 print(is_palindrome("a")) # True
16
17
18
```

Output:



```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C:\Users\abhin\OneDrive\Desktop\AI assisted coding\8.5.py/t3.py
True
True
False
True
True
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>
```

Observation: AI-generated tests helped identify edge cases like spaces, punctuation, and case differences. String normalization techniques were applied to ensure accurate palindrome detection. The function successfully handled empty strings and single-character inputs.

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

- Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.

- Methods:

- o deposit(amount)

- o withdraw(amount)

- o get_balance()

Example Assert Test Cases:

```
acc = BankAccount(1000)
```

```
acc.deposit(500)
```

```
assert acc.get_balance() == 1500
```

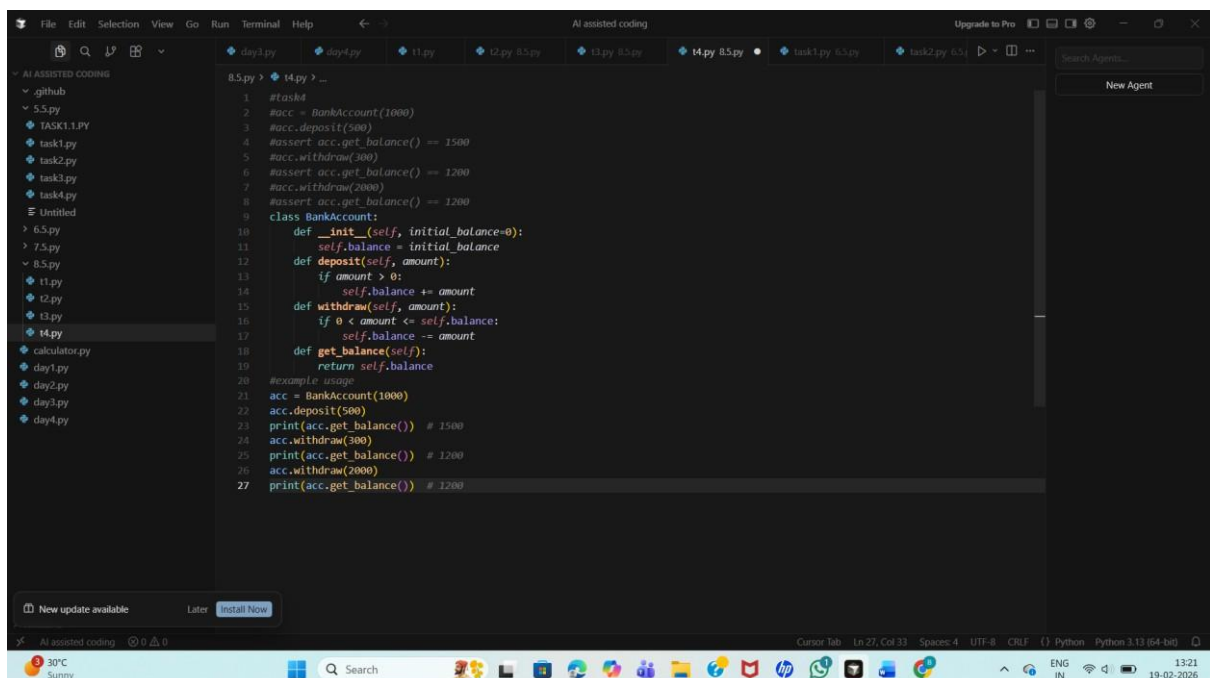
```
acc.withdraw(300)
```

```
assert acc.get_balance() == 1200
```

Expected Output #4:

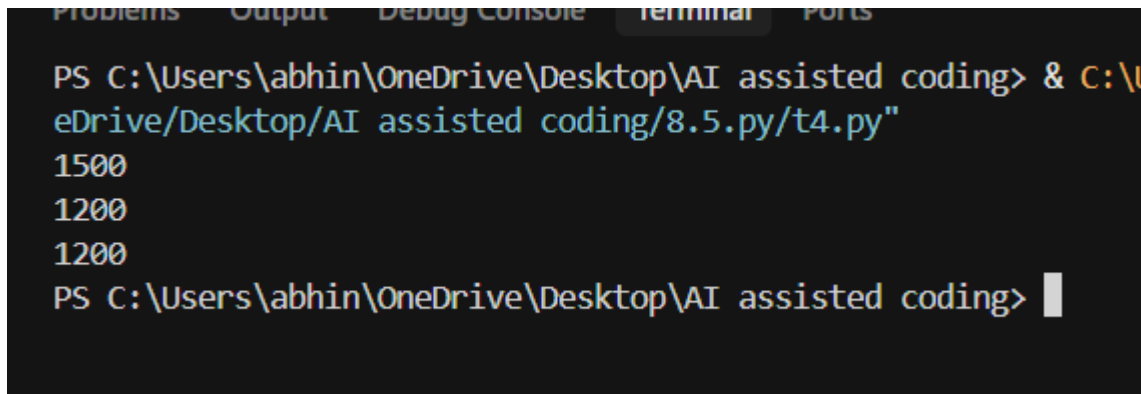
- Fully functional class that passes all AI-generated assertions.

Code:



```
1 #task4
2 #acc = BankAccount(1000)
3 #acc.deposit(500)
4 #assert acc.get_balance() == 1500
5 #acc.withdraw(300)
6 #assert acc.get_balance() == 1200
7 #acc.withdraw(2000)
8 #assert acc.get_balance() == 1200
9
10 class BankAccount:
11     def __init__(self, initial_balance=0):
12         self.balance = initial_balance
13     def deposit(self, amount):
14         if amount > 0:
15             self.balance += amount
16     def withdraw(self, amount):
17         if 0 < amount <= self.balance:
18             self.balance -= amount
19     def get_balance(self):
20         return self.balance
21
22 #example usage
23 acc = BankAccount(1000)
24 acc.deposit(500)
25 print(acc.get_balance()) # 1500
26 acc.withdraw(300)
27 print(acc.get_balance()) # 1200
28 acc.withdraw(2000)
29 print(acc.get_balance()) # 1200
```

Output:

A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. The prompt is 'PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>'. The command entered is '& C:\Users\abhin\OneDrive\Desktop\AI assisted coding\8.5.py/t4.py"'. The output shows three lines: '1500', '1200', and '1200'. The prompt is repeated at the bottom: 'PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>'.

```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C:\Users\abhin\OneDrive\Desktop\AI assisted coding\8.5.py/t4.py"
1500
1200
1200
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>
```

Observation: AI-generated test cases helped design object-oriented methods before implementation. The class correctly handled deposits, withdrawals, and balance retrieval. Test-driven development ensured correct behavior and reduced logical errors in financial operations

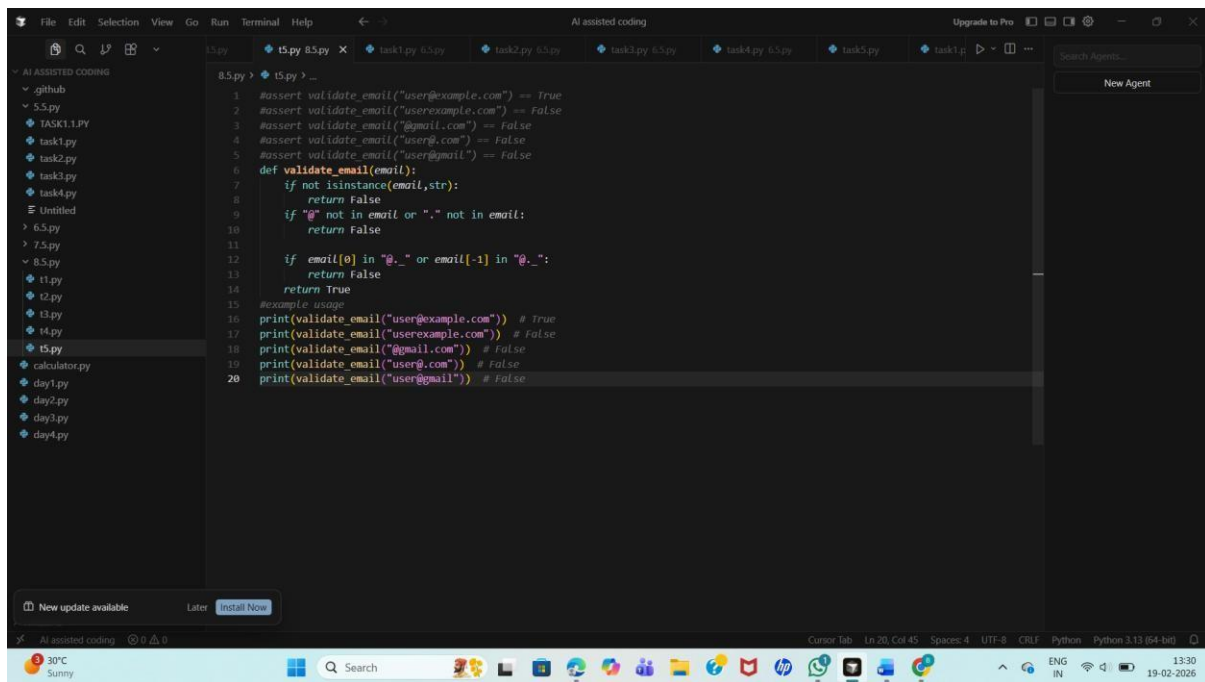
Task Description #5 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.
- Requirements:
 - o Must contain @ and .
 - o Must not start or end with special characters.
 - o Should handle invalid formats gracefully.

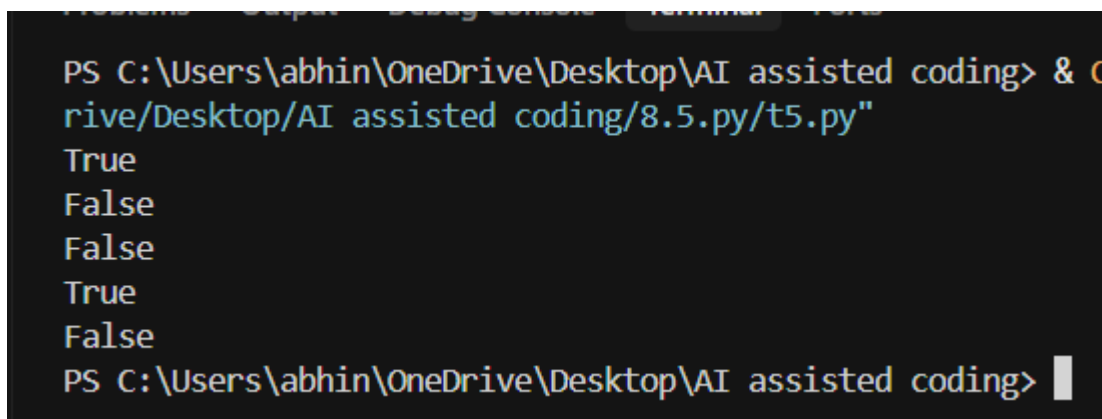
Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
```

code:



Output:



Observation: AI test cases guided the validation rules for email format. The function correctly checked for required symbols and invalid formats. Edge cases such as missing symbols and improper placement were handled effectively, improving data validation reliability.